CS336 作业 3：缩放定律（Scaling Laws）

原文：Stanford CS336 作业说明（英文原版）

本文档为英文原版作业说明的非官方中文翻译，仅供学习参考；如与原文有歧义，请以英文原文为准。

目录

1 作业概述 **2**

2 缩放定律回顾 **3**

2.1 基于 IsoFLOPs 轮廓的缩放定律 3

3 构建缩放定律 **4**

3.1 训练运行细节 5

3.2 训练 API 5

3.3 排行榜 9

1 作业概述

在本作业中，你将获得一些关于语言模型缩放定律（scaling laws）的动手实践经验。

问题设定

你负责训练 ClosedAI 的下一代语言模型。训练这个模型将耗费一个小国的 GDP，并融化一座南极冰架，所以你要确保把它做对。具体来说，你会得到一个固定的算力预算（以实际运行时间（wall-clock time）衡量）。你的目标是训练出训练损失最低的模型（即计算最优（compute-optimal）的模型）。为了实现这一目标，你需要弄清楚如何在对（1）训练更大的模型与（2）在更多 token 上训练之间做出最佳权衡。

缩放定律从经验上把语言模型的训练损失与模型规模以及训练所用的算力联系起来，常被用来预测这种权衡关系 [J. Kaplan et al., 2020; J. Hoffmann et al., 2022]。在本作业中，你将构建缩放定律，为一个在 48 个 B200 小时上训练的模型估计计算最优的模型规模及其对应的超参数。你不需要亲自训练这些模型，而是向一个训练 API 发起查询，提供（1）模型超参数（即 Transformer 层数、嵌入大小、注意力头数、批量大小、学习率、训练 token 数等）以及（2）实验的最长实际运行时间；API 将返回用相应超参数训练得到的验证损失。用于拟合缩放定律的预算是额外的 12 个 B200 小时（占我们大规模运行 FLOPs 预算的 25%）。该训练 API 接受范围很宽的超参数取值，你的部分职责就是弄清楚应该探索搜索空间的哪些部分，以便高效地利用缩放定律预算。

代码是什么样的

本作业说明发布在 GitHub 上：

github.com/stanford-cs336/assignment3-scaling

请使用 Git 克隆该仓库。如果有任何更新，我们会通知你，届时你可以通过 `git pull` 获取最新版本。

如何提交

你将把以下文件提交到 Gradescope：

- `writeup.pdf`：完整描述你拟合缩放定律、并用它预测给定 FLOPs 预算下最优模型规模的方法。报告应当详细到足以复现你的结果。
- `code.zip`：包含你为拟合缩放定律并计算估计值所编写的全部代码。

此外，请把你预测的最优超参数和预测的最终验证损失提交到 API。作业成绩的一部分将取决于你预测的最优模型的表现。

关于 AI 工具的声明

AI 可以完全自主地完成作业的许多部分。这会让人更难深入地投入课程材料并从中学习。

我们允许使用 AI 工具回答高层次的概念性问题，或提供低层次的编程文档（例如函数签名和库 API）。但是，不允许使用 AI 工具实现作业的任何部分。这既包括编码智能体（例如 Cursor Agents、Codex、Claude Code），也包括 AI 自动补全（例如 Cursor Tab、GitHub Copilot）。使用 AI 智能体时，请确保它使用所提供的 `AGENTS.md` 文件；使用聊天机器人时，也应当附上你的提示词。

我们强烈建议你在完成作业时在 IDE 中禁用 AI 自动补全（例如 Cursor Tab、GitHub Copilot）（不过非 AI 的自动补全（例如函数名补全）是完全没有问题的）。往届同学指出，禁用 AI 自动补全让他们更容易深入地投入到课程材料中。

完整的 AI 政策请参见这份文档。

2 缩放定律回顾

我们将回顾 Chinchilla 论文 [J. Hoffmann et al., 2022] 中拟合缩放定律的一种方法。核心问题是：给定一个将用于训练大型语言模型的算力预算 C，怎样的超参数选择（模型规模、训练 token 数等）能带来最低的训练损失？主要挑战在于：如何把较小规模下完成的实验外推到更大的规模。对于你在作业第二部分中的工作，欢迎借鉴其他参考文献中的思想，例如 J. Kaplan et al. [1] 和 G. Yang et al. [3]。

2.1 基于 IsoFLOPs 轮廓的缩放定律

回顾一下：在包含 D 个 token 的数据集上训练一个有 N 个参数的 Transformer，其算力预算近似为 $C = 6ND$。J. Hoffmann et al. [2] 中拟合缩放定律的 IsoFLOPs 方法如下：对每个算力预算 C，在该预算下训练不同规模 N 的语言模型（数据规模为 $D = \frac{C}{6N}$），得到最终训练损失 L。

这样会得到一组使用了相同 FLOPs 数 C_i、但模型规模 N_{ij} 各不相同的运行。从经验上看，J. Hoffmann et al. [2] 观察到：在固定算力预算 C_i 下，最终训练损失 L_{ij} 与模型规模 N_{ij} 之间呈二次关系。一个直观的解释如下：当 N_{ij} 极小时，无论我们投入多少算力，模型都无法拟合数据，因此这个区域的最终训练损失很高。随着模型规模增大，最终训练损失平滑下降，直到某一点：模型开始变得过大，在 C_i FLOPs 之内我们负担不起足够的梯度步数来有效训练它（举一个极端的例子：当 $N_{ij} \to \infty$ 时，哪怕只走一步梯度都会超出算力预算 C，训练因此会在非常高的损失处停止）。寻找缩放定律的方法将包括：为每个 C_i 确定最优的模型规模和数据规模，然后拟合一个幂律，在给定目标算力 C（通常大于先前运行所用的预算）时预测 N 和 D。

为了拟合幂律，我们使用如下数据点：对每个预算 C_i，利用使用该预算的那组运行（即它的「IsoFLOPs 轮廓」），找到使训练损失最小的模型规模 $N_{\text{opt}}(C_i)$。这一过程给了我们一系列数对 $\langle C_i, N_{\text{opt}}(C_i)\rangle$（对 D_{opt} 也有一个类似的结果）。我们用这些数据点拟合幂律 $N_{\text{opt}} \propto C^a$ 与 $D_{\text{opt}} \propto C^b$，并用它们把计算最优的模型规模和数据规模外推到我们的目标算力预算。

题目（`chinchilla_isoflops`）：IsoFLOPs 缩放定律（5 分）

请编写一个脚本，使用一组训练运行的最终训练损失，复现上文描述的 IsoFLOPs 缩放定律拟合方法。对于本题，请使用文件 `data/isoflops_curves.json` 中给出的（合成的）训练运行数据。该文件包含一个 JSON 数组，其中每个元素是一个描述某次训练运行的对象。下面是前两次运行，用来说明格式：

```
[
    {
        "parameters": 49999999,
        "compute_budget": 6e+18,
        "final_loss": 7.192784500319437
    },
    {
        "parameters": 78730505,
        "compute_budget": 6e+18,
        "final_loss": 6.750171320661809
    },
    ...
]
```

在拟合缩放定律时，`scipy` 包（尤其是 `scipy.optimize.curve_fit` ）可能会有用，不过你也可以使用任何你喜欢的曲线拟合方法。虽然 J. Hoffmann et al. [2] 对每个 IsoFLOP 轮廓拟合一个二次函数来寻找其最小值，但我们建议你更简单地直接把每个算力预算下训练损失最低的那次运行作为最小值。

(a) 给出你外推得到的计算最优模型规模，以及你得到的 $\langle C_i, N_{\text{opt}}(C_i)\rangle$ 数据点。对于 10^{23} FLOPs 的预算，你预测的最优模型规模是多少？10^{24} FLOPs 呢？

交付要求：画一张图，展示你的模型规模随算力预算变化的缩放定律，图中标出用于拟合缩放定律的数据点，并外推到至少 10^{24} FLOPs。然后用一句话给出你预测的最优模型规模。

(b) 给出你外推得到的计算最优数据集规模，以及训练运行中的 $\langle C_i, D_{\text{opt}}(C_i)\rangle$ 数据点。对于 10^{23} 和 10^{24} FLOPs 的预算，你预测的最优数据集规模分别是多少？

交付要求：画一张图，展示你的数据集规模随算力预算变化的缩放定律，图中标出用于拟合缩放定律的数据点，并外推到至少 10^{24} FLOPs。然后用一句话给出你预测的最优数据集规模。

3 构建缩放定律

在本节中，你将在固定算力预算下，使用通过我们的训练 API（见第 3.2 节）查询得到的训练运行来拟合缩放定律。你的首要目标是选出能在 48 个 B200 小时上取得最低验证损失的模型规模和超参数；你还要预测该模型将取得的验证损失。为了给预测出的最优模型规模设定超参数，我们建议你分析超参数在较小规模设定下对验证损失的影响。请务必在开始之前仔细规划你的运行——一旦超出 12 个 B200 小时的缩放定律预算，训练 API 将拒绝后续请求。

3.1 训练运行细节

训练 API 在 B200 GPU 上运行真实的训练任务。当你提交一个实验时，它会进入一个共享队列。排队中和运行中的实验都会按其完整的 `max_runtime_seconds` 占用你的 12 小时缩放定律预算。当一个实验完成或失败时，API 会按该实验实际上报的运行时间重新计算你的预算，上报时间会被截断（clip）到至少 1 秒、至多 `max_runtime_seconds` 。因此，如果你为一个在 12 分钟内完成的运行预留了 30 分钟，未使用的 18 分钟会退回到你的剩余预算。如果一次运行超时，则按 `max_runtime_seconds` 计费。

每个训练任务会运行 `n_evals` 个分块（chunk）。API 每个分块训练 `total_train_tokens / n_evals` 个 token，每个分块结束后在验证集上评估，并在 `status.val_losses` 中报告验证损失序列。对于已完成的实验，最终验证损失就是该列表的最后一项。对于超时失败的实验，API 可能返回部分验证损失，但这些并不是已完成的运行。

数据顺序是固定的。训练数据是分词后的 DCLM 数据，最多 500B 个 token。不存在数据轮次（data epoching）。对每个训练配置，API 都按相同且确定的训练 token 顺序消费数据。验证集同样使用 2^{18} 个验证 token。最终的排行榜验证损失会使用更多 token。`model_seed` 字段只控制模型初始化，不控制数据顺序。

模型架构与作业 1 大体相同，只有少数细微差别。我们在作业仓库中提供了该模型的代码供你参考：https://github.com/stanford-cs336/assignment3-scaling。训练代码仅作为参考提供。你只应通过 API 来训练模型。

这些模型在分词后的 DCLM 数据上训练，词表大小为 32K，上下文长度为 512。API 支持 AdamW 和 SGD，两者都使用预热余弦（warmup-cosine）学习率调度。下面的 API 示例使用 AdamW，权重衰减 0.01，梯度裁剪 1.0，预热比例 0.05，最终学习率比例 0.1。训练使用在 512 token 序列上取平均的下一 token 交叉熵损失。API 模型中没有 dropout。

3.2 训练 API

你将使用这个训练 API 来查询你想运行的缩放定律实验的最终验证损失。你的 API key 将是你的 8 位数字 Stanford 学号（例如 06123456）。注意，你必须接入 Stanford 网络才能查询该 API，因此你可能需要使用 VPN。作为基本的完整性检查，你应该能打开 http://hyperturing.stanford.edu:8000/docs 上的 API 文档页面，并使用 `/budget` 端点验证你的 API key 有效（详见下文）。训练 API 包含以下端点：

所有 API 请求都应在 `X-API-Key` 请求头中带上你的 API key：

```
>>> API_BASE_URL = "http://hyperturing.stanford.edu:8000"
>>> API_KEY = <YOUR_API_KEY_HERE>
>>> headers = {"X-API-Key": API_KEY}
>>> requests.get(f"{API_BASE_URL}/budget", headers=headers).json()
{'used_seconds': 30.0, 'remaining_seconds': 43170.0, 'total_budget_seconds': 43200.0}
```

你需要把实验提交到队列，轮询它们的状态，并在实验完成后读取验证损失。

`POST /submit`

给定一个训练配置，把一次训练运行加入队列并返回其实验 id。请求体应当是一个 JSON 对象，包含以下键：

- `architecture_config` ：描述 Transformer 架构的 JSON 对象。它的键包括 `attention_bias` , `head_dim` , `hidden_size` , `intermediate_size` , `num_attention_heads` , `num_hidden_layers` , `num_key_value_heads` , `rms_norm_eps` , `rope_theta` , `tie_word_embeddings` , `dtype` 和 `vocab_size` 。其中 `dtype` 字段必须是 `"float32"` 或 `"bfloat16"` 。
- `optimizer_config` ：描述优化器的 JSON 对象。我们支持 AdamW 和 SGD 两种配置；下面的示例使用 AdamW。
- `train_batch_size` ：整数训练批量大小。
- `val_batch_size` ：整数验证批量大小。
- `n_evals` ：训练过程中要运行的验证评估次数。
- `total_train_tokens` ：训练 token 总数。
- `max_runtime_seconds` ：为该实验预留的最长实际运行时间。该值必须介于 1 秒到 12 小时之间。
- `model_seed` ：用于初始化模型的随机种子。

API 把 `seq_len` 固定为 512，把 `n_validation_tokens` 固定为 $2^{18} \approx 262k$，并校验若干一致性条件。例如，`hidden_size` 必须等于 `num_attention_heads * head_dim` ，`num_attention_heads` 必须能被 `num_key_value_heads` 整除，`total_train_tokens` 必须能被 `* train_batch_size` 整除。

响应是一个 JSON 对象，包含以下键：

- `experiment_id` ：排队实验的整数 id。
- `budget_summary` ：包含 `used_seconds` , `remaining_seconds` 和 `total_budget_seconds` 的 JSON 对象。

如果你把相同的训练配置提交两次，API 会返回 409 响应，并且不会预留额外预算。如果 `max_runtime_seconds` 超过你的剩余预算，API 会返回 400 响应。

查询该 API 时，请发起 HTTP POST 请求：

```
>>> import requests
>>> API_BASE_URL = "http://hyperturing.stanford.edu:8000"
>>> headers = {"X-API-Key": <YOUR_API_KEY_HERE>}
>>> training_config = {
...     "architecture_config": {
...         "attention_bias": False,
...         "head_dim": 64,
```

```
...         "hidden_size": 448,
...         "intermediate_size": 1280,
...         "num_attention_heads": 7,
...         "num_hidden_layers": 9,
...         "num_key_value_heads": 7,
...         "rms_norm_eps": 1e-6,
...         "rope_theta": 1_000_000,
...         "tie_word_embeddings": False,
...         "dtype": "bfloat16",
...         "vocab_size": 32_000,
...     },
...     "optimizer_config": {
...         "lr_scheduler": {
...             "peak_value": 3e-4,
...             "final_lr_frac": 0.1,
...             "warmup_frac": 0.05,
...             "init_value": 0.0,
...         },
...         "weight_decay": 1e-2,
...         "beta1": 0.9,
...         "beta2": 0.95,
...         "eps": 1e-8,
...         "eps_root": 1e-8,
...         "grad_clip_norm": 1.0,
...     },
...     "train_batch_size": 128,
...     "val_batch_size": 32,
...     "n_evals": 16,
...     "total_train_tokens": 1_048_576,
...     "max_runtime_seconds": 30.0,
...     "model_seed": 0,
... }
>>> requests.post(f"{API_BASE_URL}/submit", headers=headers, json=training_config).json()
{'experiment_id': 1, 'budget_summary': {'used_seconds': 30.0, 'remaining_seconds': 43170.0,
'total_budget_seconds': 43200.0}}
```

GET /budget

给定一个 API key，返回你的实验预留或已使用的实际时间预算。响应是一个 JSON 对象，包含以下键：

- `used_seconds` ：你已提交的实验使用或预留的实际时间总秒数。
- `remaining_seconds` ：你的缩放定律预算中剩余的实际时间秒数。
- `total_budget_seconds` ：缩放定律预算的实际时间总秒数。

查询该 API 时，请发起 HTTP GET 请求：

```
>>> requests.get(f"{API_BASE_URL}/budget", headers=headers).json()
{'used_seconds': 30.0, 'remaining_seconds': 43170.0, 'total_budget_seconds': 43200.0}
```

GET /experiments

返回用你的 API key 提交的所有实验的列表。每个实验包含以下键：

- `experiment_id` ：由 `/submit` 返回的实验 id。
- `training_config` ：为该实验提交的训练配置。
- `status` ：实验当前的状态。

`status.status_type` 字段是以下取值之一：

- `"queued"` ：实验正在等待运行。
- `"running"` ：实验正在运行。状态中包含 `val_losses` ，即到目前为止观察到的验证损失。实验进入队列后会处于 running 状态。这其中包含启动时间和抢占（preemption），因此你的实验显示为 running 的时间可能超过你指定的最长运行时间。
- `"completed"` ：实验已结束。状态中包含 `used_runtime_seconds` 、`val_losses` 和 `completed_at` 。
- `"failed"` ：实验失败或超时。超时失败会包含 `partial_val_losses` ，你可以查看它们；而失败的实验要么已经崩溃，要么已经超时。

查询该 API 时，请发起 HTTP GET 请求：

```
>>> requests.get(f"{API_BASE_URL}/experiments", headers=headers).json()
[{'experiment_id': 1, 'training_config': {...}, 'status': {'status_type': 'queued',
'queued_at': '...'}}]
```

GET /experiment/{experiment_id}

返回单个实验，其响应格式与 `/experiments` 返回的条目相同。使用该端点轮询已提交的实验，直到其 `status.status_type` 变为 `"completed"` ：

```
>>> requests.get(f"{API_BASE_URL}/experiment/1", headers=headers).json()
{'experiment_id': 1, 'training_config': {...}, 'status': {'status_type': 'completed',
'val_losses': [3.1, 2.9, ...], ...}}
```

对于已完成的实验，请使用 `status.val_losses` 中的最后一项作为该次运行的最终验证损失。

POST /final_submission

提交你预测的最优超参数和预测的最终验证损失。请求体应包含：

- `training_config` ：你预测的最优训练配置。
- `predicted_final_loss` ：你为该配置预测的最终验证损失。

你可以向该端点重复提交；最新一次提交会替换之前的那次。

```
>>> final_submission = {
...     "training_config": training_config,
...     "predicted_final_loss": 2.75,
... }
>>> requests.post(f"{API_BASE_URL}/final_submission", headers=headers,
json=final_submission).json()
{'training_config': {...}, 'predicted_final_loss': 2.75, 'submitted_at': '...'}
```

GET /final_submission

返回你当前的最终提交；如果你还没有提交过，则返回 `None` ：

```
>>> requests.get(f"{API_BASE_URL}/final_submission", headers=headers).json()
{'training_config': {...}, 'predicted_final_loss': 2.75, 'submitted_at': '...'}
```

3.3 排行榜

现在，你将使用训练 API 为 48 个 B200 小时的运行选择一个模型配置。目标是最小化这个最终模型的验证损失。

题目（`scaling_laws` ）：构建缩放定律排行榜（50 分）

构建一个缩放定律，并用它选出你预期在 48 个 B200 小时上取得最低验证损失的模型规模和超参数。你还需要预测该模型将取得的验证损失。为了构建你的缩放定律，你将使用我们的训练 API 查询各种实验配置（见第 3.2 节）的最终验证损失；用于拟合缩放定律的实验查询不得超过 12 个 B200 小时。这是一条由 API 强制执行的硬性上限。

交付要求：一份排版好的报告，完整描述你拟合缩放定律的思路和方法。此外，报告还应描述你如何使用缩放定律预测给定 FLOPs 预算下的最优模型规模，以及你的预测值。报告应包含你为何做出特定设计决策的说明，并且描述应详细到足以复现你的方法和结果。

对于你在 48 个 B200 小时运行下报告的超参数，我们基本不设任何限制。所有运行都在单张 B200 上进行。

为了帮助你入门，我们建议你至少思考以下问题。你的报告应针对下面每个因素补充说明决策是如何做出的：

- 在 12 个 B200 小时的固定缩放定律预算下，你是如何决定查询哪些运行的？

- 你是如何拟合缩放定律的？请描述你使用的具体方法。特别地，熟悉 J. Kaplan et al. [1] 和 J. Hoffmann et al. [2] 中使用的方法很可能会有帮助。
- 你的缩放定律对实验数据的拟合效果如何？
- 对于给定的 48 个 B200 小时 FLOPs 预算，你的缩放定律预测的最优模型规模是多少？预测的损失是多少？
- 如果你要用预测出的最优参数量训练一个模型，你会使用什么超参数？要估计给定模型超参数配置下的非嵌入（non-embedding）参数量，请使用 $12\, n_{\text{layer}} d_{\text{model}}^2$。

除报告之外，请将（1）你预测的最优超参数和（2）模型的验证损失提交到 API。作业成绩的一部分将取决于你预测的最优模型的表现。

参考文献

[1] J. Kaplan *et al.*, “Scaling Laws for Neural Language Models.” 2020.

[2] J. Hoffmann *et al.*, “Training Compute-Optimal Large Language Models.” 2022.

[3] G. Yang *et al.*, “Tensor Programs V: Tuning Large Neural Networks via Zero-Shot Hyperparameter Transfer.” 2022.